

Homework Assignment: Git Fundamentals and Workflow Verification

This assignment is designed to introduce you to professional Git workflows using the command line. You will create an empty repository, update the repository structure, work on a feature branch, merge your work back into the main branch, and generate a single proof file that captures both your Git history and repository structure.

Learning Objectives

- Initialize and structure a Git repository
- Create and work on a feature branch
- Merge a branch back into main
- Generate verifiable Git activity artifacts

Assignment Overview

You will create an empty repository and then clone it, create a standard project directory structure on a feature branch, merge that branch back into main, and generate a single text file that shows both your Git commit history and the repository structure. You will then submit this file to Blackboard.

Part 0: Repository Creation

Create an empty repository on Github via the UI. Name the repository whatever you would like to. Instantiate it with a *python* .ignore and README.md file (these are options you can select during repository creation). If you don't have a Github account, you will have to create one before completing this assignment. You will need your username and email you use to create your account later in this process.

Part 1: Repository Setup

On your local machine or in a cloud environment (you can complete this assignment from Google Colab), clone this repository and navigate into it with command line commands. Verify that the repository contains a README and .gitignore file and that your working tree is clean.

*Note – if you are using Collab to complete this, you will need to add an “!” before commands to tell Collab to run the command as a command line command. When changing directories, use ‘%’ instead of ‘!’ (i.e. %cd directory_name)

Part 2: Create a Feature Branch

Create a new branch named **feature/project-structure**. Make sure you name the branch as instructed. All work in the next section must be completed on this branch.

Part 3: Create Project Structure

Create the following directory structure:

```
data/  
notebooks/  
src/  
tests/  
docs/
```

Once the above directories have been created, add placeholder files as needed so Git tracks these directories. Run the following commands (only after you have created the directory structure above):

```
touch data/.gitkeep  
  
touch notebooks/example.ipynb  
  
touch src/__init__.py  
  
touch tests/__init__.py  
  
touch docs/README.md
```

Once completed, stage (add) and commit your changes with a clear commit message.

Part 4: Merge Back to Main

Switch back to the main branch and merge feature/project-structure into main. This merge should create a merge commit.

Part 5: Generate Git Proof File

From the root of your repository, generate a single text file named **firstname_lastname_git_activity.txt** that contains (*replace firstname and lastname with your actual name*):

1. Your full Git commit history across all branches
2. Your repository directory structure (excluding the .git directory)

Use the following commands to satisfy 1 and 2 above (these commands will work with MacOS, Linux, or Windows Bash):

```
git log --oneline --graph --decorate --all > firstname_lastname_git_activity.txt  
echo "\n==== REPOSITORY STRUCTURE ====\n" >> firstname_lastname_git_activity.txt  
find . -path "./.git" -prune -o -print >> firstname_lastname_git_activity.txt
```

Submission Instructions

Submit ONLY the file `firstname_lastname_git_activity.txt` to Blackboard.